

main.py × +

Console

I/O

AI Agent

133 ms ✓

```
1 def adicionar(item, lista=[]):
2     lista.append(item)
3     return lista
4
5 print(adicionar(1))
6 print(adicionar(2))
7
8 # (a) [1]
9 #     [1, 2]
10 # (b) argumento padrão mutável: o [] é criado só uma vez, na definição da função,
11 #     então é a mesma lista em todas as chamadas (conserto: lista=None)
```

```
[1]
[1, 2]
```

Main.java

```
1 public class Main {
2     static void zera(int[] v, int n) {
3         v[0] = 0;
4         n = 0;
5     }
6
7     public static void main(String[] args) {
8         int[] v = {5, 5}; int n = 5;
9         zera(v, n);
10        System.out.println(v[0] + " " + n);
11        // (a) 0 5
12        // (b) passagem por valor: n é cópia do int, então n = 0 não afeta o original
13        //      v é cópia da referência, então v[0] = 0 altera o mesmo array
14    }
15 }
```

Console

I/O

AI Agent

881 ms

0 5

main.py × +

```
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
3
4 # (a) [2, 2, 2]
5 # (b) closure / late binding: a lambda guarda a variável i, não o valor,
6 #     então quando chama o i já é 2
```

Console

I/O

AI Agent

137 ms ✓

[2, 2, 2]

C Main.c x +

Console

I/O

AI Agent

178 ms ✓

```
1  #include <stdio.h>
2
3  int contador(void) {
4      static int n = 0;
5      return ++n;
6  }
7
8  int main() {
9      contador(); contador();
10     printf("%d\n", contador());
11     // (a) 3
12     // (b) variável static local: mantém o valor entre as chamadas
13     //      (fica na área estática, não na pilha)
14     return 0;
15 }
```

3

HelloWorld.rs ×

```
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(v);
8     println!("{:?} {:?}", v, d);
9     // (a) não compila: erro "borrow of moved value: `v`"
10    // (b) ownership/move: dobra(v) tomou a posse de v, então não dá pra usar depois
11    // (conserto: dobra(v.clone()) -> saída [1, 2, 3] [2, 4, 6])
12 }
```

Console

I/O

AI Agent

Exit 1

```
error[E0382]: borrow of moved value: `v`
--> HelloWorld.rs:8:27
```

```
6 |     let v = vec![1, 2, 3];
  |           - move occurs because `v` has type `Vec<i32>`, which does not
  |           implement the `Copy` trait
7 |     let d = dobra(v);
  |               - value moved here
8 |     println!("{:?} {:?}", v, d);
  |                       ^ value borrowed here after move
```

note: consider changing this parameter type in function `dobra` to borrow instead if owning the value isn't necessary
--> HelloWorld.rs:1:13

```
1 | fn dobra(v: Vec<i32>) -> Vec<i32> {
  |     ----- ^^^^^^^^^ this parameter takes ownership of the value
  |     |
  |     in this function
help: consider cloning the value if the performance cost is acceptable
7 |     let d = dobra(v.clone());
  |                   +++++++
```

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.

Debug this error

Light Wiki Internet

22:14
05/10/2026

main.py x +

```
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
8 # (a) UnboundLocalError: local variable 'total' referenced before assignment
9 # (b) escopo (regra LEGB): atribuir dentro da função cria total local,
10 # então ele lê antes de existir (conserto: global total)
```

Console

I/O

AI Agent

Exit 1

Traceback (most recent call last):

```
File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ^^^^^^^^^^^^^
```

```
File "/box/main.py", line 4, in adiciona
    total = total + x
    ^^^^^
```

UnboundLocalError: cannot access local variable 'total' where it is not associated with a value

[Debug this error](#)